MIS Backend

System Architecture & Module Reference

NestJS 11 | MongoDB | TypeScript | Puppeteer | Resend | JWT

High-level documentation
Generated April 2026

Table of Contents

1. Auth — Authentication & JWT
2. User — User management
3. Role — RBAC roles
4. Permission — RBAC permissions
5. Entity — Client organizations
6. CertificationBody — CABs and Standards
7. Application — Certification applications
8. BA — Business Associates
9. Country — Reference data
10. Email — Notification service
11. Certificate — PDF generation (Puppeteer)
12. Common — Shared interfaces
13. Module Summary Matrix

Backend Modules

01 Auth

Authentication, JWT issuance, OTP-based 2FA, password management.

KEY ENDPOINTS

- **POST** `/auth/login` — Login with email/password; returns OTP prompt in production
- **POST** `/auth/verify-otp` — Verify OTP and issue JWT cookie
- **POST** `/auth/resend-otp` — Resend OTP to user's email
- **POST** `/auth/logout` — Clear authentication cookie
- **GET** `/auth/me` — Get current authenticated user
- **PATCH** `/auth/profile` — Update firstName, lastName, phone
- **POST** `/auth/change-password` — Change password

KEY SCHEMA (USERACCOUNT)

- `userId` — string, 7-char unique ID
- `email` — unique, lowercase
- `password` — bcrypt hash, `select: false`
- `role` — ObjectId ref to UserRole
- `status` — active | inactive | pending | rejected | suspended
- `twoFA` — { enabled, tempSecret, secret }

> OTP skipped in non-production. JWT expires in 1 day. EventEmitter triggers email on login.

02 User

CRUD operations for system users, role assignment, permission enforcement.

KEY ENDPOINTS

- **POST** `/user` — Create user (manage:users / user:create)
- **GET** `/user/get-all` — Paginated list with search
- **GET** `/user/:id` — Get with role + reportingManager populated
- **PATCH** `/user/:id` — Update fields (firstName, lastName, role, status, password, etc.)

> Unique `userId` auto-generated as 7-char alphanumeric. Passwords hashed with bcryptjs. Permissions enforced per operation.

03 Role

RBAC role definitions, permission assignment, role hierarchy.

KEY ENDPOINTS

- **GET** `/role` — All roles unpaginated
- **GET** `/role/get-all` — Paginated
- **POST** `/role` — Create new role
- **PATCH** `/role/:id` — Update role

KEY SCHEMA (USERROLE)

- `role` — string, unique (e.g., "BA Manager", "Scope Manager")
- `permissions` — ObjectId[] (Permission refs)
- `reportingRole` — ObjectId (UserRole self-ref for hierarchy)
- `cabCode` — string[] CAB restrictions
- `region` — India | Overseas
- `type` — default | custom

04 Permission

RBAC permission management, category-based organization.

KEY ENDPOINTS

- **GET** `/permission` — All permissions
- **GET** `/permission/get-all` — Paginated
- **POST** `/permission` — Create
- **PATCH** `/permission/:id` — Update

KEY SCHEMA (PERMISSION)

- `name` — string, unique, lowercase (e.g., "user:create", "certificate:view")
- `category` — string (e.g., "User Management")
- `type` — default | custom
- `status` — active | inactive

05 Entity

Organizations / companies seeking ISO certification — the primary client entity.

KEY ENDPOINTS

- **POST** `/entity/create` — Create entity (BA or direct client)
- **GET** `/entity/get-all` — Paginated list with BA filter
- **GET** `/entity/:id` — Get with related applications populated
- **PATCH** `/entity/:id` — Update by entity_id

KEY SCHEMA (ENTITY)

- `entity_id` — auto-generated unique alphanumeric
- `entity_name` — required, unique, max 100 chars
- `main_site_address[]` — { street, city, state, country, postal_code }
- `additional_site_address[]` — extended address with separate_legal_entity sub-doc
- `primary_contact_person` — { name, designation, email, mobile_number }
- `status` — draft | submitted | verified | approved | rejected
- `certificateStatus / scopeStatus / qualityStatus / docStatus` — multi-stage tracking
- `isDirectClient` — boolean (BA-driven vs. self-served)

> Email verification states (by-pass / pending / verified). Emits `entity:created` event for email notifications. Auto-generates unique `name_slug`.

06 CertificationBody (CAB)

Manages Certification Bodies (CABs) and certification standards with jurisdiction support.

KEY ENDPOINTS

- **POST** `/certificationbody` — Create CAB
- **GET** `/certificationbody` — Paginated list
- **GET** `/certificationbody/:id` — Get with standards & jurisdictions populated
- **PATCH** `/certificationbody/:id` — Update; syncs standards array
- **POST** `/certificationbody/standard` — Create standard
- **GET** `/certificationbody/standard/all` — Paginated standards
- **PATCH** `/certificationbody/standard/:id` — Update standard

KEY SCHEMA

CertificationBody	CertificationStandard
`cabCode` — 3-char unique uppercase (TCU, TSI, GAU, GAI)	`mssCode` — 3-char unique alphanumeric
`cbCode / cbName` — Certification body	`standardCode` — e.g. ISO 9001
`abCode / abName` — Accreditation body	`version` — e.g. 2015, 2025
`cabJurisdictions[]` — Country refs	`predecessor / successor` — version chain refs
`standards[]` — Standard refs	`certificationBodies[]` — back-refs

> Standards support version chains. CAB jurisdiction is checked against entity's country on application creation. Standards are bidirectionally synced with CABs.

07 Application

ISO certification applications. Multi-stage approval (scope, quality, certificate, BA, finance) and draft / final certificate lifecycle.

KEY ENDPOINTS

- **POST** `/application` — Create (validates CAB jurisdiction vs entity address)
- **GET** `/application` — Paginated list with cabCode filter
- **GET** `/application/draft` — Draft apps for scope review (scopeStatus filter)
- **GET** `/application/:id` — Single application
- **PATCH** `/application/:id` — Update scope, audit days, IAF code, scopeStatus

KEY SCHEMA (APPLICATION)

- `cab_code` , `standards[]` , `entity` ref, `scope`
- `primary / secondary_certificate_language`
- `certificateStatus` — proceed | completed | hold | suspended | active | expired | etc.
- `scopeStatus` — pending | rejected | transfer | completed
- `qualityStatus` — pending | rejected | proceed | completed
- `baStatus / baManagerStatus / clientStatus` — applied | final
- `audit1, audit2, iaf_code, auditor_leader_name` — scope review fields
- `draftCertificate[] / finalCertificate[]` — versioned, multi-language URLs
- `certificate_number, initial_issue, valid_until` — issuance metadata

> Multi-stage parallel approval flow (scope → quality → BA → finance → certificate). Emits `application.draft.approved` event when scopeStatus becomes "completed", which triggers the Certificate module.

08 BA (Business Associate)

Business associates who work with CABs on entity audits and certifications.

KEY ENDPOINTS

- **POST** `/ba` — Create BA (creates User + CabBA together)
- **PATCH** `/ba/:id` — Update BA (User + CabBA fields synced)
- **GET** `/ba/get-all` — Unpaginated, with search
- **GET** `/ba/get-all-paginated` — Paginated with search
- **GET** `/ba/get-all-cabBa` — All CAB-BA associations
- **GET** `/ba/:id` — Get with cab details populated

KEY SCHEMA

User (BA role)	CabBA
`username, userId, email, phone`	`contact_name, registration_number`
`cab` — CabBA ref	`registration_authority`
`cabCode[]` — TCU, TSI, GAU...	`address` — { street, city, state, country, postal_code }
`status` — active | inactive | pending | suspended	`currency, gst, certificateLanguage`
`mouStatus` — approved | pending | uploaded | rejected	`cab[]` — CB sub-docs with standards & rate cards

09 Country

Reference data for country / jurisdiction lookups.

KEY ENDPOINTS

- **GET** `/country` — All countries
- **GET** `/country/languages` — Supported certificate languages (13 incl. EN, HI, ES, FR, AR, ZH, PT, DE, IT, JA, KO, RU, TR)

KEY SCHEMA (COUNTRY)

- `name` — unique
- `code` — 3-char uppercase, unique
- `status` — active | inactive

> Used by CAB jurisdiction validation and entity address lookups.

10 Email

Notification service backed by Resend; OTP delivery and entity verification emails.

EVENT LISTENERS

- `user:login` — Generates 6-digit OTP (10-min TTL), stores in Redis cache, sends email
- `entity:created` — Sends entity verification email with verification link

KEY FUNCTIONS

- `handleUserLogin()` — OTP email on login
- `sendMailForEntity()` — Entity creation confirmation
- `verifyOtp()` — Validate OTP from cache

> Email templates differ by CAB (Guardian, TNV variants) and environment (dev / prod).

11 Certificate (new)

Generates draft certificate PDFs from HTML using Puppeteer; triggered automatically when an application's draft is approved.

ARCHITECTURE

- HTML template `templates/draft.html` rendered with application + entity data
- Puppeteer (headless Chromium) converts HTML → PDF at A4 (794×1123px)
- Background image picked by `cab_code` : TCU.png / TSI.png / GAU.png / GAI.png from `templates/darft/`
- Output saved to `generated-certificates/` ; URL stored in application's `draftCertificate[]` array
- Versioning supported (v1, v2...) and per-language variants

TRIGGER FLOW

- Scope manager approves draft via `PATCH /application/:id` with `scopeStatus: "completed"`
- Application service emits `application.draft.approved`
- `CertificateEventListener` calls `generateDraftCertificate(applicationId)`
- Service builds HTML, runs Puppeteer, persists PDF, updates DB

DYNAMIC BEHAVIOR

- Entity name and scope font sizes auto-scale based on text length
- Standard names dynamically inserted in certify line ("...the Quality Management System of...")
- Draft certificates show `XXXXXXXXXX` placeholders for cert no, dates, etc.; only IAF code & revision no use real values when present
- Watermark "Draft copy must be returned within 15 days" overlaid diagonally

> Built with: `puppeteer ^24.x` , NestJS event emitter, MongoDB push update on application document.

12 Common

Shared interfaces, DTOs, guards, and utilities across modules.

KEY FILES

- `auth-request.interface.ts` — Express Request extension with authenticated user payload (userId, role, permissions[])
- `paginated-query.dto.ts` — Common DTO: page, limit, search, scopeStatus, cabCode
- `paginated-response.dto.ts` — Standard paginated response shape
- JWT Auth Guard — Validates JWT from cookies; populates `req.user`

Module Summary Matrix

Module	Primary Schema	Key Status Fields	Notable
Auth	UserAccount	active / inactive / suspended	JWT cookie, OTP 2FA
User	UserAccount	active / pending / rejected	bcrypt, RBAC enforced
Role	UserRole	active / inactive	Self-referencing hierarchy
Permission	Permission	active / inactive	Granular, categorized
Entity	Entity	draft → approved + cert/scope/quality status	Email verification, slug
CertificationBody	CAB + Standard	active / inactive / expired	Jurisdictions, version chains
Application	Application	multi-stage (scope, quality, BA, finance)	Triggers cert generation
BA	User + CabBA	active, mouStatus	Bi-directional sync
Country	Country	active / inactive	Reference data
Email	—	—	Resend, event-driven
Certificate	—	—	Puppeteer, event-driven, NEW
Common	—	—	Auth, pagination, guards

End of Documentation

This is a high-level overview. For exact field definitions, see the module schema files.

Generated from MIS Backend repository using Puppeteer.